

MARTIN Gabriel

Cahier des Charges Fonctionnel

Application de gestion de collection cinématographique

Jalon 1 — Janvier 2026

I. PRÉSENTATION D'ENSEMBLE DU PROJET

1. Présentation du contexte

Le projet vise à développer une application web permettant aux particuliers de centraliser et d'organiser leur consommation cinématographique.

Dans un contexte où l'offre de films est dispersée sur de multiples plateformes (Cinéma, Netflix, TV, DVD), les amateurs de cinéma peinent à garder une trace de leur historique. Cette plateforme offrira aux utilisateurs la possibilité de rechercher des films via une base de données (exemple : TMDB), de gérer une liste d'envies (« Watchlist »), de valider les films vus et de leur attribuer une note personnelle.

L'administrateur pourra quant à lui gérer les utilisateurs inscrits et modérer la plateforme.

Problématiques identifiées :

- Listes de films éparpillées (notes téléphones, post-it, mémoire défaillante).
- Difficulté à se souvenir si un film a déjà été vu ou s'il a été apprécié.
- Perte de temps à rechercher des informations sur un film (synopsis, date de sortie) sur différents sites.
- Absence d'outil simple et centralisé respectant la confidentialité des données.

Impact utilisateur :

- Gain de temps dans le choix d'un film (consultation rapide de la Watchlist).
- Satisfaction de construire une « collection » virtuelle organisée.
- Accès centralisé aux informations techniques des films (données importées automatiquement).

2. Objectifs de l'application

2.1. Objectifs principaux

- Développer une application web moderne en architecture Headless (API Symfony + Front React).
- Consommer une API tierce pour récupérer les données des films (titre, affiche, synopsis) sans saisie manuelle.
- Permettre la gestion du cycle de vie d'un film dans la collection : Ajout en « À voir », bascule en « Vu », Notation.

2.2. Objectifs secondaires

- Offrir une interface Responsive Design (adaptée mobile et desktop).
- Sécuriser les données utilisateurs (RGPD) et l'accès à l'application (Authentification robuste).
- Mettre en place un tableau de bord utilisateur permettant de filtrer rapidement sa collection (Vus / Non vus).

3. Les cibles adressées par le site

L'application s'adresse à deux types d'acteurs distincts :

Utilisateurs finaux (Front-Office) :

- Le cinéphile régulier : Adulte 25–45 ans, habitué des plateformes de streaming (Netflix, Disney+), souhaitant ne plus perdre la trace de ses films vus ou à voir. Il utilise son smartphone autant que son ordinateur.
- Le « Binge-watcher » : Utilisateur intensif consommant plusieurs films ou séries par semaine, qui a besoin d'un historique clair et d'un système de notation personnel pour s'y retrouver.
- Le collectionneur : Amateur de DVD ou Blu-ray souhaitant centraliser sa bibliothèque physique et numérique dans un outil unique.

Administrateur du site (Back-Office) :

- Gestionnaire de la communauté chargé de la maintenance des comptes utilisateurs et de la modération de la plateforme (droit à l'oubli, RGPD).

4. Objectifs quantitatifs

- Performance : Temps de chargement des pages inférieur à 1 seconde grâce à l'architecture SPA (Single Page Application).
- Qualité du code : Couverture de tests unitaires (Back-end) supérieure à 70 % pour garantir la fiabilité de la gestion des données.
- Scalabilité : Capacité de gérer des listes utilisateurs de nombreux films sans ralentissement.
- Disponibilité : Disponibilité de l'application de 99 % via une conteneurisation Docker stable.

5. Périmètre du projet

- Zone géographique : Accessible depuis n'importe quel navigateur web, sans restriction géographique.
- Langue : Interface en français. Les données des films sont récupérées en français via le paramètre de langue de l'API TMDB (fr-FR). L'application n'est pas prévue pour être multilingue dans ce périmètre.
- Plateforme : Application Web (SPA) accessible via navigateur Desktop et Mobile.

II. PÉRIMÈTRE FONCTIONNEL DÉTAILLÉ

1. Module Utilisateur (Front-Office)

Authentification :

- Inscription.
- Connexion (Récupération de Token sécurisé JWT).
- Déconnexion.

Recherche :

- Barre de recherche connectée en temps réel à l'API TMDB (Recherche par titre).

Fiche Film :

- Affichage des détails d'un film (Titre, Date, Synopsis, Casting, Affiche).

Gestion de Collection (CRUD Métier) :

- Bouton « Ajouter à ma liste ».
- Gestion du statut : « À voir » (Watchlist) ou « Vu ».
- Notation : Attribuer une note (étoiles de 1 à 5).
- Suppression : Retirer un film de sa collection.

Filtres :

- Voir uniquement la Watchlist ou uniquement les films Vus.

2. Module Administration (Back-Office)

- Gestion des membres : Visualisation de la liste des inscrits.
- Modération : Possibilité de supprimer un compte utilisateur (Droit à l'oubli / RGPD).

III. EXIGENCES ET CHOIX TECHNIQUES

1. Architecture Logicielle — Analyse comparative et choix retenu

Deux approches ont été évaluées pour ce projet :

Critère	Symfony Full-Stack (Twig)	Architecture Headless (API + React)
Complexité	Faible (tout en Symfony/Twig)	Plus élevée (gestion CORS, JWT)
Expérience utilisateur	Rechargements de page complets	Navigation fluide sans rechargement (SPA)
Séparation des responsabilités	Couplage front/back	Découplage total Front / API
Scalabilité	Limitée	Élevée (API réutilisable, front indépendant)
Pertinence pédagogique	Couvre les bases	Couvre React, API REST, JWT, CORS
Adapté au projet ?	Suffisant pour un back-office	Recommandé pour une SPA grand public

→ Choix retenu : Architecture Headless (API Symfony + Front React).

Justification : L'application cible un public large, avec une expérience utilisateur fluide comme priorité (navigation sans rechargement, consultation rapide de la Watchlist). React est la bibliothèque front-end la plus utilisée sur le marché et permet de répondre à ces exigences. Symfony est retenu côté back-end pour sa robustesse, sa conformité aux standards PHP (PSR) et sa maîtrise au sein de la formation. La séparation des responsabilités entre l'API et l'interface facilite également la testabilité et l'évolution future de l'application.

Angular a été écarté au profit de React : Angular impose une structure plus rigide adaptée aux grandes équipes, alors que React offre plus de flexibilité pour un projet individuel. La communauté React est également plus large, ce qui facilite l'accès à des ressources d'apprentissage.

2. Socle Technique

- Langage Back : PHP 8+
- Framework Back : Symfony (version LTS récente) — expose uniquement une API REST au format JSON.
- Framework Front : React (dernière version stable) — SPA gérant l'affichage et la navigation fluide.
- Base de données : MySQL (Relationnel). Schéma normalisé via Doctrine ORM pour gérer les relations Utilisateurs ↔ Films.
- API Externe : The Movie Database (TMDB) — récupération automatique des données films.
- Conteneurisation : Docker et Docker Compose pour orchestrer l'API, la Base de données et le serveur Front.

- Versionning : Git (GitHub/GitLab) avec stratégie de branches (Main / Develop / Features).

3. Contraintes et Enjeux

3.1. Contraintes Temporelles et Organisationnelles

- Durée du projet : 6 mois, de janvier 2026 à juin 2026.
- Jalons livrables : 6 jalons mensuels impératifs, chacun validant une étape clé (Conception, Base de données, Architecture, Bêta, Release finale).
- Gestion du temps : Le projet étant individuel, une rigueur stricte sera appliquée pour assumer simultanément les rôles de Chef de projet, Développeur Back/Front et DevOps.

3.2. Contraintes Réglementaires (RGPD)

- Conformité : L'application manipulant des données personnelles (email, mot de passe, préférences), elle se conformera au RGPD.
- Droits utilisateurs : Mise en place des mécanismes permettant le droit à l'oubli (suppression de compte), la rectification des données et le consentement explicite lors de l'inscription.

3.3. Contraintes Techniques Imposées

Les exigences techniques suivantes sont imposées par le cahier des charges de la formation et devront être respectées tout au long du projet :

- Tests automatisés : Mise en place de tests unitaires (PHPUnit côté back-end) et de tests fonctionnels ou d'intégration (ex : tests d'API via Symfony WebTestCase ou Postman). L'objectif est une couverture supérieure à 70 % des classes métier.
- Intégration Continue (CI) : Configuration d'une pipeline CI (GitHub Actions ou GitLab CI) déclenchée à chaque push pour exécuter automatiquement les tests et valider le build. Cela garantit la détection rapide de régressions.
- Déploiement Continu (CD) : Mise en place progressive d'un volet CD pour automatiser le packaging Docker (build d'image) et, si possible, le déploiement sur un environnement cible.
- Sécurité OWASP : L'application devra se prémunir contre les failles critiques (Injection SQL via l'ORM Doctrine, XSS via l'échappement React/Twig, CSRF, hachage des mots de passe Bcrypt/Argon2, contrôle d'accès par rôles).
- Architecture MVC / N-Tiers : Séparation stricte des responsabilités (Contrôleurs, Services métier, Entités/Repositories, Interface React).

3.4. Risques Majeurs et Hypothèses

- Dépendance API Tierce : Le fonctionnement du site repose sur l'API TMDB.
 - Risque : Indisponibilité du service ou changement de la politique tarifaire / quotas.
 - Hypothèse : L'API TMDB restera disponible en accès gratuit pour les volumes de développement.

- Courbe d'apprentissage (Architecture Headless) : La séparation Back-end (Symfony) et Front-end (React) ajoute une complexité technique (gestion des CORS, authentification JWT) par rapport à une architecture monolithique classique.
- Délai court : Le projet étant individuel sur 6 mois, le risque de ne pas implémenter toutes les fonctionnalités souhaitées dans le temps imparti est identifié. Les fonctionnalités secondaires (filtres avancés, pagination) seront traitées en priorité basse.

3.5. Critères de Succès

Le projet sera considéré comme une réussite si les conditions suivantes sont remplies à l'issue du Jalon 6 :

- Fonctionnel : L'ensemble des fonctionnalités décrites (Recherche, CRUD Collection, Filtres) sont opérationnelles.
- Performance : Temps de réponse de l'API inférieur à 500 ms et fluidité de navigation sur le Front (SPA).
- Qualité : Passage réussi des tests automatisés (CI/CD) et absence de failles de sécurité critiques (OWASP). Couverture de tests > 70 %.
- Déploiement : L'application est entièrement conteneurisée et déployable via une simple commande Docker.